PROJECT DOCUMENTATION / OCTOBER 2026

CraftLife

Turn everyday progress into an RPG adventure.

An offline-first, Minecraft-inspired personal development app bringing habits, goals, learning, health, money management, and RPG progression into one place.

PROJECT TYPE Independent software project	**PLATFORM** Windows desktop (public release)
STATUS Beta / actively developed	**CORE STACK** Python · PyQt6 · React · TypeScript · SQLite

SOURCE REPOSITORY

github.com/Hellowww-02/CraftLife

01 / Project overview

A reader-friendly introduction to the product and its purpose.

CraftLife is a personal development application built around one simple idea: everyday actions can feel more rewarding when progress is visible. Instead of treating habits, study sessions, workouts, and personal goals as disconnected checklists, CraftLife connects them to an RPG-style character, experience points, rewards, and progression.

The project follows an offline-first approach. Core personal data is stored locally in SQLite, while optional cloud services support capabilities such as multi-device synchronization and online social interactions. The public repository describes a Windows desktop app with a React interface running inside a PyQt6 shell.

Project links

- **Source code:** GitHub repository
- **Latest release:** CraftLife Releases
- **Audience:** people who want a more engaging way to organize routines, personal goals, and everyday progress.
- **Status:** Beta; features and implementation may change as development continues.

Project mission

> *Make personal growth feel less like repetitive administration and more like a meaningful journey—while keeping the experience practical enough for everyday use.*

02 / The problem and the approach

Challenge	CraftLife's approach
Habit tracking can feel repetitive.	Habits, dailies, and quests connect to visible progression such as XP, levels, streaks, and rewards.
Personal organization is spread across tools.	The app brings together productivity, learning, health, finances, notes, music, and calendar features.
Users want control over their data.	The desktop app uses a local SQLite database and can operate offline for core local workflows.
Progress can be hard to notice.	Dashboards, charts, streaks, achievements, and summaries help users review activity.
A personal tool can feel solitary.	Optional cloud features add social functions such as friends, chat, guilds, leaderboards, and sync.

Gamification is the presentation layer; the underlying purpose remains helping people manage real-life activities.

03 / Main features

Grouped by the kind of real-life activity each area supports.

Character & progression

Dashboard with XP, HP, MP, gold, levels, streaks, avatar classes, talents, titles, achievements, and a Year Wrapped-style report.

Habits & tasks

Positive and negative habits, recurring dailies, one-time quests, streak tracking, folders, templates, and task reordering.

RPG gameplay

Shop and equipment, crafting, enchanting, pets, pet training and buffs, boss battles, guild bosses, redeem codes, and leaderboards.

Health & fitness

Workout logging, rep charts, food and meal logging, nutrition estimates, health metrics, water goals, and a Pomodoro timer.

Money management

Income and expense tracking, budgets, debts and installments, savings, investments, subscriptions, and multi-currency display.

Notes & learning

Organized notes, attachments, LaTeX preview, AI-assisted learning chat, quizzes, flashcards, summaries, study guides, and generated learning artifacts.

Calendar & reminders

Calendar notes, reminders with sounds, recurring reminders, and Indonesian public holiday data.

Music

Local music library, playlists, mini-player, synced lyrics, and optional music downloads.

Social & cloud

Optional friends, chat, guilds, PvP challenges, shared couple-oriented space, notifications, cloud linking, and sync.

Customization & maintenance

Multiple themes, accessibility options, settings, import/export, database maintenance, and in-app update support.

Feature note

This list reflects the public README for the v1.7.4 release line. Some features depend on configuration, internet access, or external services; availability may differ by installation.

04 / How the experience works

A typical user flow:

- **1. Set up:** open CraftLife, configure a local profile, and choose preferences.
- **2. Plan:** add habits, recurring dailies, quests, study tasks, workouts, or other activities.
- **3. Take action:** complete activities and record relevant details in the appropriate tracker.
- **4. See progress:** review XP, streaks, charts, character stats, achievements, and summaries.
- **5. Keep improving:** use the feedback to adjust routines and continue progressing.
- **6. Optional online layer:** connect cloud services if multi-device sync or social functionality is wanted.

This is a conceptual overview, not a guarantee that every feature follows the exact same interaction pattern.

05 / Technical architecture

Based on the public repository README and listed entry-point files.

CraftLife combines a Python desktop layer with a web-based interface. Python handles the desktop shell and local application services; React provides the main UI; SQLite stores local data. A local API connects the interface to

backend operations. Optional Supabase services provide cloud capabilities.

Layer	Technology / component	Responsibility
Desktop shell	Python · PyQt6 · PyQt6 WebEngine	Starts the desktop app and hosts the React interface.
Frontend	React · TypeScript · Vite · Tailwind CSS	Renders pages and user interactions.
Local API	api_server.py and related modules	Connects frontend requests to local application logic.
Local data	SQLite · database.py	Stores core personal and application data on-device.
Optional cloud	Supabase · cloud/sync modules	Supports cloud and social functionality where configured.
AI / external tools	Gemini integration and utility modules	Enables optional AI-assisted learning and selected online utilities.

Simplified request path

```
React interface
      |
      v
PyQt6 WebEngine desktop container
      |
      v
Local API (documented local endpoint: 127.0.0.1:8765)
      |
      v
Python application services
      |
      v
SQLite database  <---->  optional Supabase services
```

This is a simplified diagram. Individual features may use different service modules, and cloud-dependent actions require network access and valid configuration.

06 / Key technologies and repository map

Technology / file	Role in the project
Python	Desktop application logic, local services, integrations, and data operations.
PyQt6 / PyQt6 WebEngine	Desktop window and embedded web UI container.
React + TypeScript	Component-based interface and page-level UI.
Vite + Tailwind CSS	Frontend build tooling and styling workflow.
SQLite	Local database for core offline-first operation.
Supabase	Optional cloud services such as sync and online/social features.
MainPyQt6.py	Primary desktop application entry point listed in the repository.
api_server.py	Local API entry point listed in the repository.
database.py	Database module listed in the repository.
web/	Frontend application and JavaScript/TypeScript dependencies.
supabase/	Cloud configuration, migrations, and related assets.
requirements.txt / package.json	Python and frontend dependency manifests.

Repository layout may evolve. Check the current README and dependency files before following setup instructions.

07 / Running the project

For most readers, the simplest way to try CraftLife is to use the latest published release rather than building from source.

Windows user route

- Open the latest release page.
- Download the release ZIP and extract it.
- Run **`CraftLife.exe`**.
- Back up personal data before major updates. The README documents the database at `%APPDATA%\CraftLife\craftlife.db`.

Developer route

```
git clone https://github.com/Hellowww-02/CraftLife.git
cd CraftLife
pip install -r requirements.txt
cd web
npm install
npm run build
cd ..
python MainPyQt6.py
```

These are the source-build commands documented by the repository README. A development environment also needs compatible Python and Node.js/npm versions; check the current requirements before installing.

Internet dependency

The README describes core local use as offline-capable. Internet access is needed for optional features such as cloud synchronization, AI chat, podcast voices, and music downloads. Exact requirements vary by feature and configuration.

08 / Design principles

Offline-first

Core local workflows should remain useful without requiring a cloud account.

Data ownership

Local personal data is stored in a database on the user's device, with backup/export options documented in the app.

Progress through feedback

XP, streaks, charts, and achievements provide visible feedback for repeated effort.

One application, multiple life areas

The project aims to reduce context switching by combining several trackers in one workspace.

Bilingual access

The public README describes Indonesian and English UI coverage.

Incremental development

Release notes show continued refinement of performance, usability, accessibility, and feature behavior.

09 / Current status and limitations

The public README identifies v1.7.4 as the latest release at the time this document was prepared. CraftLife remains a developing project, so its feature set and implementation details can change between releases.

Important context for readers

- CraftLife is desktop-first in its current public release documentation. Android work and cross-device plans should be described as development goals unless a release explicitly confirms availability.
- Cloud, AI, social, and download features may require external accounts, credentials, network access, or provider availability.
- This document summarizes public project information. It is not a security audit, performance benchmark, or independent verification of every feature.
- Screens, commands, dependency versions, and configuration may change; the repository README and release notes remain the source of truth.

Development direction

The broader direction is to make CraftLife more dependable and accessible while preserving its core identity: an RPG-inspired system for managing real-life progress. Planned platform expansion should be distinguished from features confirmed in the current Windows release.

10 / Project profile

CraftLife is developed as an independent personal software project. The creator has shaped the product concept, built features, iterated on the interface, and maintained the codebase, using AI-assisted development as part of the workflow.

Skills demonstrated by the project

- Product thinking: translating a personal-development idea into feature areas and user flows.
- Desktop development: building a Python-based Windows application.
- Frontend engineering: integrating a React and TypeScript UI into a desktop shell.
- Data management: using SQLite and maintaining database behavior across updates.
- Integration work: connecting optional cloud and AI services while preserving local workflows.
- Iterative maintenance: documenting releases, refining interface behavior, and addressing bugs.

This describes work represented by the project; it does not imply that every feature is production-ready or independently tested.

11 / Quick glossary

Term	Plain-language meaning
Offline-first	The app is designed to remain useful locally even when the internet is unavailable.
RPG gamification	Game-like progression—levels, XP, rewards, achievements—makes progress more visible.
SQLite	A lightweight database stored as a file on the device.
React	A JavaScript library used to build an interface from reusable components.
PyQt6 WebEngine	A desktop component that displays web-based UI inside a Python application.
Supabase	A hosted backend platform used for optional cloud-related functionality.
API	A defined way for one part of the application to request work or data from another part.

12 / References and update note

Prepared on 21 October 2026 using the public CraftLife repository and README available at the time of preparation.

Primary references

- GitHub repository — source tree, project files, and release documentation.
- README.md — overview, features, stack, setup instructions, and release notes.
- Latest release page — published builds and release details.

For the most accurate current information, refer to the repository before redistributing this document. This PDF is intended for portfolio uploads, project showcases, and readers who want to understand CraftLife without first exploring the source code.